NBA Win Market – Darren Lin

A win-probability prototype for NBA games. The project uses a dual-branch fusion model implemented in PyTorch: a custom Transformer encoder reads pregame and live natural-language commentary, and a small neural network reads team/player statistics. The two branches are fused to output the home team's win probability. A Streamlit web app demonstrates the model live, combining the trained model's output with a score/time adjustment and a pretrained sentiment model that reacts to typed-in commentary.

Research question

Team statistics give a reasonable baseline for predicting NBA outcomes, but they don't capture information that only exists as language: an injury, an ejection, a player heating up. This project tests whether a Transformer-based text encoder can add independent signal on top of a statistical baseline, and demonstrates that idea live via a rule-based prototype that combines the trained model with real-time commentary.

Data

Source: Historical NBA Data and Player Box Scores (Kaggle, author: eoinamoore).

- Raw tables used: Games.csv (73,279 rows), TeamStatistics.csv (146,560 rows), PlayerStatisticsExtended.csv (838,803 rows).
- Fixed a real data quality issue: a subset of TeamStatistics rows had a corrupted teamId of 0, recovered via a Games-table lookup on home/away team IDs.
- After cleaning and filtering to complete cases: 36,085 games in the final training table.
- All rolling features (5-game and 10-game averages) are computed with .shift(1) before .rolling(), so the current game is excluded from its own features (no label leakage).
- Train/validation split is time-based: games before 2024-01-01 train the model, games on/after that date validate it.

Data is not re-uploaded to this repository; download it via `kagglehub` as shown in `nba_project_clean_pipeline.ipynb`.

Model architecture

- **Text branch:** token embeddings + sinusoidal positional encoding + multi-head self-attention (custom Transformer encoder, vocabulary size 1,495, built from the project's own training corpus).
- **Stats branch:** a small feed-forward network over 14 numeric features — for each team (home and away), win percentage and average points scored over both the last 5 and last 10 games, plus the recent performance values of that team's top scorer, top playmaker, and top rebounder.
- **Fusion:** branch outputs are concatenated and passed through a classifier head that outputs the home team's win probability.

- Trained with the Adam optimizer (lr = 1e-3) and binary cross-entropy loss, 5 epochs, on 32,609 training games / 3,476 validation games.

Results

Main model (template-generated pregame text):

- Numeric-only logistic regression baseline: 63.26% validation accuracy.
- Dual-branch fusion model: approximately 62.4%–63.4% validation accuracy across the trained epochs — comparable to, not clearly better than, the baseline.
- This is an expected, honest finding: the pregame text in this dataset is generated from the same underlying statistics, so it does not yet add much independent information.

Ablation study (synthetic independent signal):

- To test the architecture's actual capacity to use language, a modified dataset was built with graded, synthetic "surprise" phrases (e.g., severe injury reports, sudden lineup changes) scaled to the pregame win-percentage gap and deliberately contradicting the statistical favorite on upsets. Pretrained GloVe embeddings (92.2% vocabulary coverage) were used for the text branch.
- Ablation dual-branch model: best validation accuracy of 79.20%.
- Ablation numeric-only baseline (same features, no text signal): 63.26%.
- This confirms the core hypothesis: when the text contains independent, game-altering information not already present in the statistics, the model successfully uses it to shift its prediction. The reason the main model doesn't show this gain is the text data, not the architecture.

Testing with independently sourced commentary (rather than template-generated or synthetic text) is the natural next step to validate this on real-world data.

Live demo mechanics

The live Streamlit app combines three things at inference time:

1. The trained model's pregame probability (from stats + narrative text).
2. A rule-based score/time-remaining adjustment as the game progresses.
3. A separate, pretrained sentiment model (DistilBERT) that reads any commentary typed into the live feed and adjusts the probability toward the team it favors.

This is a prototype for testing the idea of combining statistics with live natural-language signals — it is not a claim that the model was trained directly on thousands of real commentary examples.

Setup and running the app

This project uses uv for dependency management.

```
# Install dependencies
uv sync

# Run the live demo app
uv run streamlit run app.py
```

Required files in the project root (produced by running the training notebook `nba_project_clean_pipeline.ipynb` once):

- trained_model.pt
- vocab.json
- standardization_stats.json
- calibration.json
- latest_team_stats.csv

Known limitations

- Real-time commentary sources such as ESPN, AP, or the official NBA Stats API typically require paid licensing or have rate limits that were not practical within this project's timeline. The live demo currently uses scripted or template-generated commentary as a stand-in for a real feed.
- The main model's training text is template-generated from the same statistics used in the numeric branch, which limits how much independent signal it can contain; the ablation study demonstrates the architecture can use independent signal when it's actually present in the text.

Next steps

- Collect independently sourced commentary (real news or play-by-play text) to rigorously test the text branch's contribution on real-world data.
- Continue cleaning up documentation and reproducibility.